



LÓGICA DE PROGRAMACIÓN

Docente: Carlos Andrés Arévalo Torres

Estudiante: Julio Raul De Loor Veloz

31 de mayo del 2026

“Juego del Ahorcado”



Introducción



En esta actividad se realizó el análisis y diseño inicial de un software antes de comenzar la programación.

El programa seleccionado fue el **“Juego del Ahorcado”**.



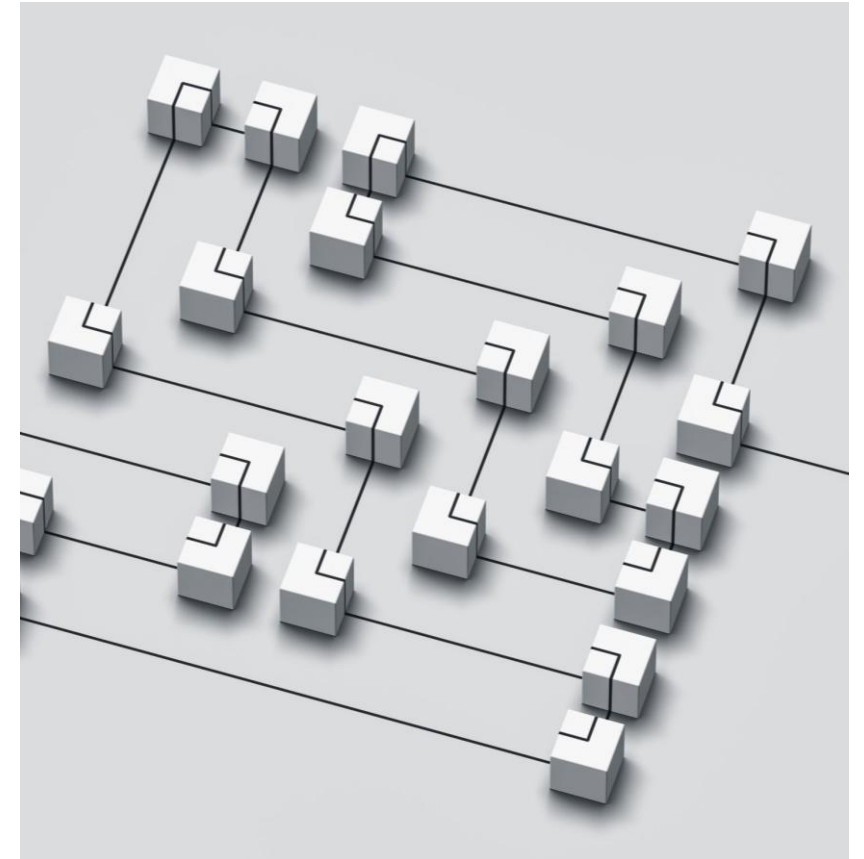
El objetivo principal es diseñar los diagramas funcionales y la arquitectura del sistema para comprender cómo funcionará el programa y cómo interactuará el usuario con él.

DIAGRAMA DE FUNCIONALIDAD

“CASOS DE USO”

Diagrama de Casos de Uso

- El diagrama de casos de uso es un tipo de diagrama de comportamiento dentro del lenguaje UML (Unified Modeling Language) que permite visualizar los requisitos funcionales de un sistema y cómo los actores (usuarios o sistemas externos) interactúan con él. Su objetivo principal es ofrecer una visión general clara y comprensible del sistema desde la perspectiva del usuario, facilitando la comunicación entre desarrolladores y clientes.



DESARROLLO DEL PROBLEMA

Para el “JUEGO DEL AHORCADO” el usuario debe iniciar sesión dentro de la aplicación para poder iniciar una partida, el programa le muestra los espacios de la palabra oculta para que el usuario/jugador piense e ingrese una letra por cada turno, el programa valida la letra y si esta dentro de la palabra oculta el programa revela la letra de la palabra y pasa al siguiente intento, pero si la letra ingresada por el jugador/usuario no es validada por el sistema dentro de la palabra el jugador/usuario pierde un intento, entonces el programa ira formando la figura hasta completar el dibujo del ahorcado, si el jugador/usuario termina la palabra antes de perder todos los intentos, entonces gana la partida. Por el contrario si el jugador/usuario no logra acertar las letras dentro de la palabra en los intentos determinados por el juego automáticamente pierde la partida.

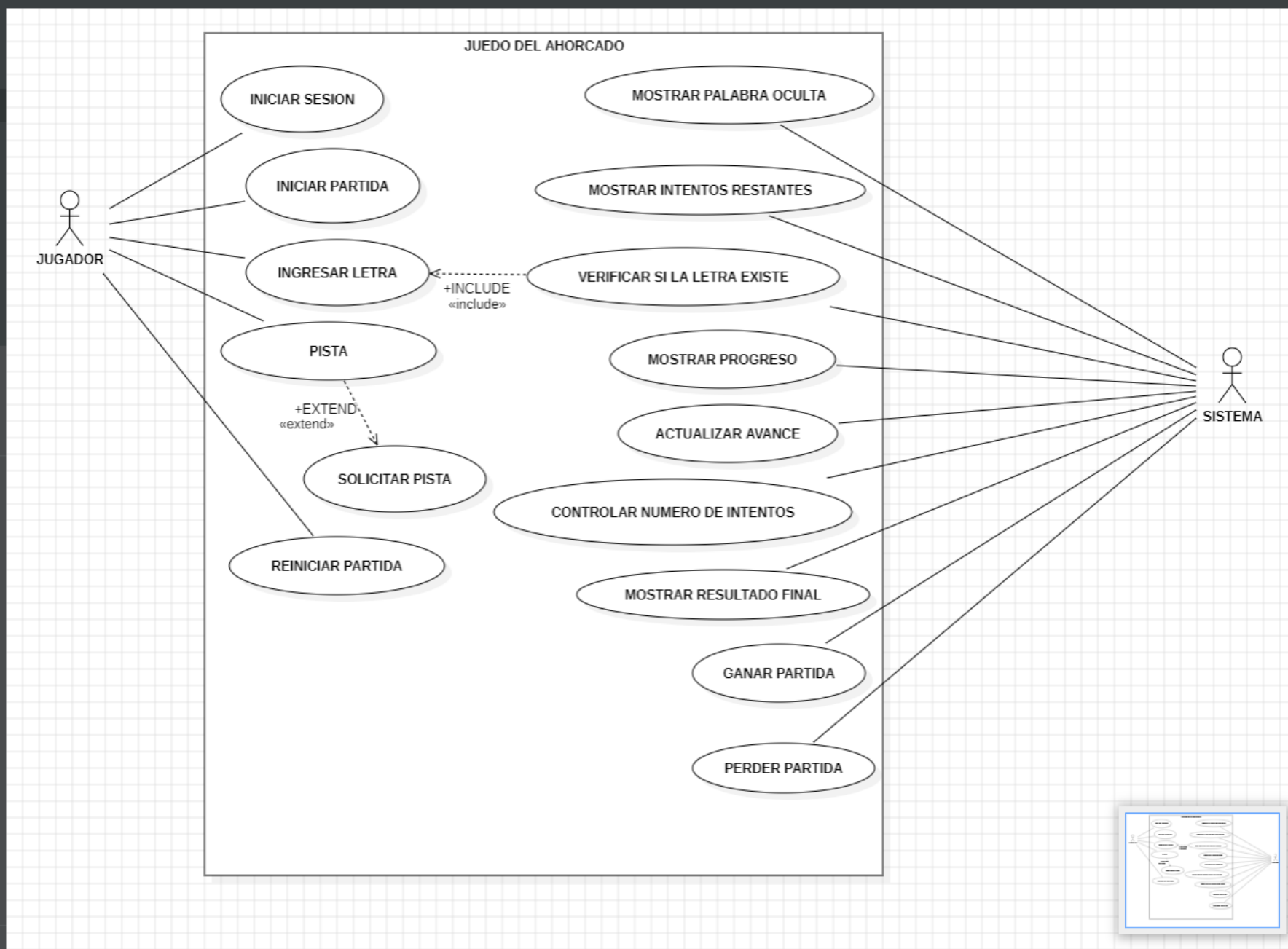
El programa también deberá mostrar los intentos restantes, actualizar los avances dentro de la palabra, mostrar el resultado final de la palabra y proporcionar una opción de pista para que el usuario decida en que momento dentro de la partida utilizarla.

 Main — Model
 UseCaseDiagram1 — Model1

Use Cases

- Package
- Use Case Subject
- Use Case
- Actor
- Frame
- Association
- Directed Association
- Generalization
- Dependency
- Include
- Extend

Annotations



- REINICIAR PARTIDA
- ▶ ♀ JUGADOR
 - ▶ ┘ (JUGADOR—INICIAR
 - ▶ ┘ (JUGADOR—INICIAR
 - ▶ ┘ (JUGADOR—PISTA)
 - ▶ ┘ (JUGADOR—REINICIA
- ▶ ♀ SISTEMA
 - ▶ ┘ (SISTEMA—MOSTRA
 - ▶ ┘ (SISTEMA—VERIFICA
 - ▶ ┘ (SISTEMA—MOSTRA
 - ▶ ┘ (SISTEMA—ACTUALIZ
 - ▶ ┘ (SISTEMA—CONTROL
 - ▶ ┘ (SISTEMA—MOSTRA
 - ▶ ┘ (SISTEMA—MOSTRA

EDITORS

-

► Properties

name

stereotype

Arquitectura en Capas - Ahorcado

CAPA DE PRESENTACIÓN

- Mostrar palabra oculta
- Mostrar letras ingresadas
- Mostrar intentos restantes
- Recibir letras del usuario

Interfaz de Usuario

CAPA DE LÓGICA

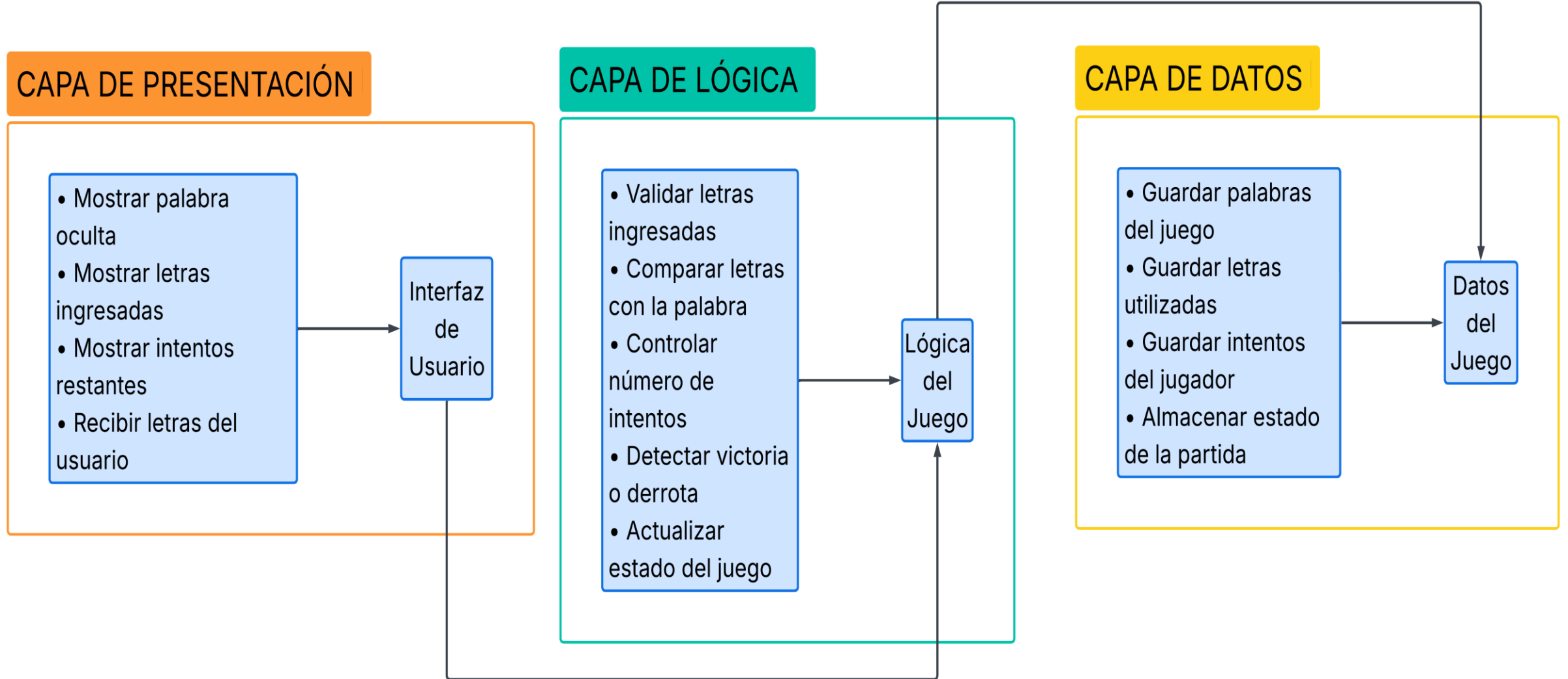
- Validar letras ingresadas
- Comparar letras con la palabra
- Controlar número de intentos
- Detectar victoria o derrota
- Actualizar estado del juego

Lógica del Juego

CAPA DE DATOS

- Guardar palabras del juego
- Guardar letras utilizadas
- Guardar intentos del jugador
- Almacenar estado de la partida

Datos del Juego



Conclusión



El análisis y diseño previo del software permite comprender mejor cómo funcionará el sistema antes de programarlo.



Mediante los diagramas funcionales y de arquitectura se logró identificar:



Las funcionalidades principales del juego



La interacción del usuario



La estructura general del sistema



Este proceso ayuda a organizar el desarrollo del software de manera más clara y ordenada.